CITS2200: Data Structures and Algorithms

Lecture 1: Unit details, objects and arrays in Python

Teaching and Learning, Administration

- Two lectures, Wednesdays 2-3 pm, Ross and Clews LT (Physics) , Fridays 11-12, Ross and Clews LT (Physics).
- One tutorial, Fridays 12-1 pm, Ross and Clews LT. **There is no tutorial in the first week**.
- The lectures and tutorials will be recorded. The Friday session will be recorded from 11 am-12:45 pm. The tutorial will start immediately after the lectures, there will be no gap.
- However, students are strongly encouraged to attend the lectures and tutorials. We will discuss lab materials and concepts from lectures in the tutorial.
- There are many drop-in sessions. **There are no labs in weeks 1, as we have to first cover some materials in the lectures. Labs will start from week 2.**
- **Lecturer:** Amitava Datta; **Facilitators:** Erick Lisangan, Gayatri Aniruddha, Hanna Zhang, Haifa Almutairi and Jason Tardi (and possibly others).
- **Consultation:** Mondays 10-11 am in Room 1.07, Computer Science Building, and also by email `amitava.datta@uwa.edu.au`. We have also a discussion forum, Amitava will post a link in LMS.

Lab enrolments

- The lab sheets will be released on LMS under a menu item 'Labs' in the left panel.
- All the tasks will be specified in a `.md` file.

Assessments

- There are three kinds of assessments in this unit.
- Some of the lab tasks are for practice, three are assessed. You can see the submission deadlines in the unit outline in LMS. You will get at least two weeks for solving the assessed labs. The submission is through LMS. Each assessed lab will contribute 10% of the total marks, for a total of 30%.
- There may be a face-to-face assessment for the labs in the last two weeks of the semester to assess your understanding. I am still working on it. The marks for the assessed labs will depend on this face-to-face assessment as well. The mark will be split between the actual submission and this face-to-face assessment.
- Two in-class multiple choice tests at 5 pm on 26/3/2026 and 30/4/2026. The venues and seating arrangements will be informed later. All students must write these two tests, as there is no possibility of any alternative arrangements or repeat tests. Each test will be worth 10% of the total marks, for a total of 20%. Any questions should be directed to Amitava.
- An end of semester examination during the June examination period. This will be a closed-book exam without any permitted materials (except for UWA approved calculators). The examination will be worth 50% of the total marks.
- The materials covered in the lectures, tutorials and labs are sufficient. However, there is a recommended text for this unit: M. Goodrich, R. Tamassia and M. Goldwasser, *Data Structures and Algorithms in Python*, 1st Edition 2013. There are electronic copies in the library.

Memory model

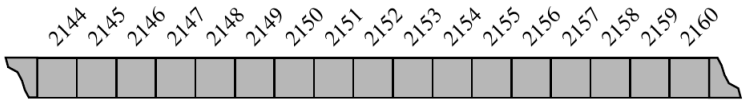

Figure 5.1: A representation of a portion of a computer's memory, with individual bytes labeled with consecutive memory addresses.

Figure 1: ©M. Goodrich : Data Structures and Algorithms in Python

- The memory of a computer is organized as bytes. Each byte has an address, an integer.
- When we store a variable in the memory, its location is a sequence of bytes. Usually one byte is not enough to store an integer or a floating point number in Python.
- We assume that any memory location can be accessed in constant time. We write this as $O(1)$ time. We will explain this later when we discuss complexity.
- What is means in simple term is, accessing any memory location takes the same amount of time by the CPU.
- This is actually not true, as we will explain later. Today's computer system use a *cache hierarchy*.

Objects in Python

- Python is an object-oriented language and classes are the basis for all data types.
- It is very easy to understand objects and how to manipulate them. We have to understand this first before we can learn about data structures.
- There are some built-in classes in Python that you have used extensively earlier, for example, `int`, `float`, `str`.
- You have seen assignment statements like `temperature = 98.6`.
- The meaning of this statement is: the left hand side declares an identifier `temperature`. The right hand side creates an object of class `float` and the *reference* of that object is stored in `temperature`.

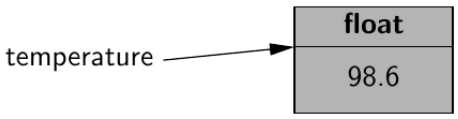

Figure 2: ©M. Goodrich : Data Structures and Algorithms in Python

Objects in Python

- Python decides what object to create by looking at the value. `98.6` is a float, so an object of the `float` class is created, and its value is given (instantiated) as 98.6.
- `temperature` is called a *reference* for this object.
- We can create another reference (also called an *alias*) for this object.
- `original = temperature`

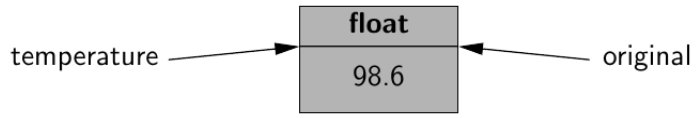

Figure 3: ©M. Goodrich : Data Structures and Algorithms in Python

Objects in Python

- If we now execute the statement `temperature = temperature + 5.0`
- The right hand side is evaluated to `103.6`, then a new float object is created and its reference is assigned to `temperature`.
- Note that, the reference `original` still holds a reference of the old float object.

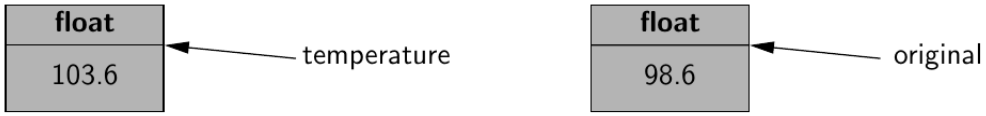

Figure 4: ©M. Goodrich : Data Structures and Algorithms in Python

Objects in Python

- Each class like `float` or `int` has *method*s that can be used to manipulate the values of the object.
- There is a special method called a *constructor*. Its purpose is to construct an object of that class.
- Instead of `temperature = 98.6`, we can write `temperature = float(98.6)`.
- The constructor has the same name as the class name (`float` in this case).
- The constructor creates an object, initializes it with the supplied value and stores in a location in the memory.

Array

- An array is a group of memory locations that store related variables, for example, a text string.
- The locations of an array are one after another, or *contiguous*.
- An array is the simplest of data structures and very useful in any programming language.

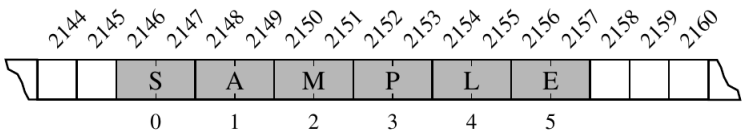

Figure 5.2: A Python string embedded as an array of characters in the computer's memory. We assume that each Unicode character of the string requires two bytes of memory. The numbers below the entries are indices into the string.

Figure 5: ©M. Goodrich : Data Structures and Algorithms in Python

- Each location of an array (two bytes in this case) is called a *cell*.
- Each location is accessed using an *index*.

Array

- Each location of an array can be accessed in constant (equal time) only if each location has the same number of bytes.
- But this is a problem, as we cannot store different objects with the same number of bytes.
- For example, if we have to store strings in an array, different strings may have different lengths, and we cannot store them with the same number of bytes.
- The advantage of using an equal number of bytes for each location is that, we can calculate the index of a location using simple calculation.
- For example, if we have an array A, and the location of $A[i]$ is byte j, we can calculate the byte number of location $A[i+4]$ by the simple calculation $j + 2 * 4$.
- Though a programmer uses a mental image (or abstraction) of an array as a sequence of two bytes in contiguous memory, the underlying storage model may be quite different.